

1. 빌드 및 배포

(1) Backend

- 기반 언어 및 프레임워크: Java 17, Spring Boot 3.4.1
- 빌드 도구: Gradle
- 실행 환경: openjdk:17-jdk-slim
- 주요 포트: 8080
- 빌드 및 실행 명령어:
`./gradlew clean build -x test`
`java -jar build/libs/api-0.0.1-SNAPSHOT.jar`

(2) Frontend

- 기반 언어 및 프레임워크: Node.js 20+, React 18, Vite, TypeScript
- 웹 서버: Nginx
- 주요 포트: 80 (Nginx 배포)
- 환경 변수 파일: .env (필수 설정: VITE_API_BASE_URL, VITE_WS_URL)
- 빌드 및 실행 명령어:
`npm install`
`npm build`
`npm run preview` 또는 `./release/{버전}/Synapse.exe` 실행

(3) Collaboration Server

- 기반 환경: Node.js (Y-Websocket)
- 역할: 실시간 동시 편집 기능 제공
- 빌드 및 실행 명령어:
`cd yjs`
`npm install`
`npm start`

(4) WebRTC Server(Signaling, TURN, STUN)

- 기반 환경: Spring(STOMP, WebSoket), KMS(Kurento Media Server), coturn(TURN, STUN)
- 역할: 동시편집 사용자 간 음성 통화 기능 제공
`# docker 이미지 활용`
`docker compose up -d kurento`

2. 외부 서비스 정보

설정파일 위치: FE(/home/ubuntu/.env.front), BE(/home/ubuntu/.env.prod)

서비스 구분	항목	설정 파일 위치 / 환경 변수명	비고
Database	Postgres	POSTGRES_DB POSTGRES_USER POSTGRES_PASSWORD	DB 스키마 생성 필요
	MongoDB	MONGO_URI	
Social Login	Google OAuth	GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET	구글 클라우드 콘솔 설정
	GitHub OAuth	GITHUB_CLIENT_ID, GITHUB_CLIENT_SECRET	깃허브 개발자 설정
AI Services	Google Gemini	GMS_API_KEY	노트 요약
	Anthropic Claude	GMS_API_KEY	코드 리뷰
Storage	AWS S3 (이미지 업로드)	AWS_ACCESS_KEY, AWS_SECRET_KEY AWS_S3_BUCKET	노트 이미지 저장
Media Server	Kurento (WebRTC)	KURENTO_URI	음성 통화 기능

3. DB 덤프 파일 (Database Dump)

postgresql

-- PostgreSQL DDL

CREATE EXTENSION IF NOT EXISTS "uuid-ossf";

-- 회원 정보

```
CREATE TABLE "member" (  
  "id" UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  "email" VARCHAR(255) NOT NULL UNIQUE,  
  "name" VARCHAR(100),  
  "theme" VARCHAR(50) DEFAULT 'light'  
);
```

-- OAuth 계정 연동

```
CREATE TABLE "oauth_account" (  
  "id" UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  "provider" VARCHAR(50) NOT NULL,  
  "provider_id" VARCHAR(255) NOT NULL,  
  "member_id" UUID NOT NULL REFERENCES "member"("id") ON DELETE CASCADE  
);
```

-- 사용자 활동 스트릭

```
CREATE TABLE "streak" (  
  "member_id" UUID NOT NULL REFERENCES "member"("id") ON DELETE CASCADE,  
  "streak_date" DATE NOT NULL,  
  PRIMARY KEY ("member_id", "streak_date")  
);
```

-- 노트 메타데이터 (실제 내용은 MongoDB에 저장되나 구조적 관계를 위해 필요)

```
CREATE TABLE "note" (  
  "id" UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  "title" VARCHAR(255),  
  "directory_path" VARCHAR(500),  
  "bookmark" BOOLEAN DEFAULT FALSE,  
  "version" BIGINT DEFAULT 1,  
  "created_by_id" UUID REFERENCES "member"("id") ON DELETE SET NULL  
);
```

-- 노트 참여자 및 권한 (협업)

```
CREATE TABLE "note_member" (  
  "note_id" UUID NOT NULL REFERENCES "note"("id") ON DELETE CASCADE,  
  "member_id" UUID NOT NULL REFERENCES "member"("id") ON DELETE CASCADE,  
  "role" VARCHAR(50) NOT NULL, -- OWNER, EDITOR, VIEWER 등  
  PRIMARY KEY ("note_id", "member_id")
```

```
);
```

```
-- 협업 초대
```

```
CREATE TABLE "invitation" (  
  "id" UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  "invitation_token" UUID NOT NULL UNIQUE,  
  "note_id" UUID NOT NULL REFERENCES "note"("id") ON DELETE CASCADE,  
  "invited_by_id" UUID NOT NULL REFERENCES "member"("id") ON DELETE CASCADE,  
  "invited_email" VARCHAR(255),  
  "invited_member_id" UUID REFERENCES "member"("id") ON DELETE SET NULL,  
  "role" VARCHAR(50),  
  "status" VARCHAR(50) DEFAULT 'PENDING'  
);
```

```
-- 마인드맵 노드 위치 정보 (개인화된 레이아웃)
```

```
CREATE TABLE "mindmap_node_position" (  
  "member_id" UUID NOT NULL REFERENCES "member"("id") ON DELETE CASCADE,  
  "note_id" UUID NOT NULL REFERENCES "note"("id") ON DELETE CASCADE,  
  "point_x" DOUBLE PRECISION,  
  "point_y" DOUBLE PRECISION,  
  PRIMARY KEY ("member_id", "note_id")  
);
```

```
-- 마인드맵 엣지 정보
```

```
CREATE TABLE "mindmap_edge" (  
  "from_id" UUID NOT NULL REFERENCES "note"("id") ON DELETE CASCADE,  
  "to_id" UUID NOT NULL REFERENCES "note"("id") ON DELETE CASCADE,  
  "member_id" UUID NOT NULL REFERENCES "member"("id") ON DELETE CASCADE,  
  PRIMARY KEY ("from_id", "to_id", "member_id")  
);
```

```
mongodb
```

```
// blocks collection
```

```
{  
  "_id": ObjectId,      // 블록 고유 ID  
  "note_id": String,    // PostgreSQL의 note.id (UUID)  
  "parent_id": ObjectId, // 트리 구조를 위한 부모 블록 ID (Optional)  
  "type": String,       // text, image, code, todo 등  
  "content": Mixed,     // 블록의 실제 내용 (텍스트 또는 JSON 데이터)  
  "properties": {       // 블록 스타일 및 설정  
    "bold": Boolean,  
    "color": String  
  },  
  "order": Number,      // 노트 내 정렬 순서  
  "created_at": Date,  
  "updated_at": Date  
}
```

4. 시연 시나리오 (Demo Scenarios)

1. 접속 및 로그인:

- 메인 페이지 접속 (/login) -> Google/GitHub 소셜 로그인 클릭 -> 로그인 완료 후 대시보드로 이동.

2. 노트 작성 (협업 기능):

- '새 노트 만들기' 클릭 -> 에디터 화면 진입 (/note/:id).
- 블록(텍스트/코드) 추가 및 실시간 타이핑 시연 (YJS 동기화 확인).
- 좌측 플로팅 영역에서 초대 링크 유효기간 설정 및 생성 가능
- 좌측 플로팅 영역에서 접속한 인원 확인 및 권한 수정 가능(최대 6명)

3. AI 기능 활용:

- 작성된 내용을 드래그하여 'AI 요약' 또는 '코드 리뷰' 요청 -> 결과 확인.

4. 마인드맵 변환:

- 작성한 노트를 기반으로 '마인드맵 뷰' 전환 (/mindmap/:id) -> 노드 확장 및 구조 시각화 확인.

5. 화상 회의 (WebRTC):

- 노트 내 사이드바에서 '음성 채널' 접속 -> 마이크/카메라 활성화 확인.